

Deep Learning Project: The Impact of Unsupervised Generative Pre-training on Deep Neural Networks

An Empirical Analysis of Classification and Synthesis on MNIST

Martin Boucher

Thomas Balsalobre

March 22, 2026

Introduction

This report investigates the performance of Deep Neural Networks (DNN) for handwritten digit classification, specifically focusing on the empirical benefits of unsupervised pre-training. Using the MNIST dataset, we evaluate a Deep Belief Network (DBN) initialization, constructed by stacking Restricted Boltzmann Machines (RBM), against standard random initialization.

The study is structured around three key variables: network depth (number of layers), layer capacity (number of neurons), and the volume of training data. Our objective is to determine how generative pre-training influences convergence and generalization in deep architectures. Additionally, a comparative analysis of generative models is provided, including RBMs, DBNs, Variational Autoencoders (VAE), Generative Adversarial Networks (GAN) and Diffusion Models, to assess their respective synthesis capabilities.

1 Execution and Deployment Guide

This project is designed as a modular hierarchy where each class extends the capabilities of the previous one to form a complete Deep Learning pipeline.

1.1 Code Architecture

The implementation is organized into three main layers:

- **RBM Class:** The fundamental building block. It handles the weights (W) and biases (a, b) between a visible and a hidden layer. It implements the Contrastive Divergence (CD-1) algorithm for unsupervised feature extraction.
- **DBN Class:** A stack of multiple RBM instances. It manages the "greedy layer-wise" pre-training where the output of layer i becomes the input for layer $i + 1$. It also includes generative capabilities through Gibbs sampling.
- **DNN Class:** Inherits from DBN. It transforms the generative stack into a classifier by adding a Softmax layer. It handles MNIST data loading, image binarization (threshold set at 127), and supervised fine-tuning via the `retropropagation` method.

1.2 Installation and Requirements

The implementation requires Python 3.8+ and the following dependencies:

- `numpy`
- `scipy`
- `matplotlib`
- `tqdm`
- `joblib`

Dependencies Bonus Part only:

- `torch`
- `diffusers`
- `transformers`
- `accelerate`
- `huggingface_hub`

1.3 How to Run the Code

The execution of the classification pipeline follows a precise sequence of function calls within the DNN class:

1. **Data Loading and Binarization:** The process begins by calling the `load_mnist` static method. This function loads the raw MNIST files and applies a fixed threshold of 127 to binarize the pixel values, a necessary step for the RBM's Bernoulli units.
2. **Architecture Initialization:** The network is initialized by instantiating the DNN class, which internally triggers `init_DNN` and `init_DBN`. This establishes the weight matrices and biases for each layer according to the defined architecture.
3. **Unsupervised Pre-training:** The `pretrain_DNN` method is then called to perform the layer-wise pre-training. This function treats the network as a stack of RBMs and trains them sequentially to extract hierarchical features without labels.
4. **Supervised Fine-tuning:** Once pre-trained, the `retropropagation` function is executed. It performs the supervised gradient descent to adjust weights across all layers, minimizing the cross-entropy loss using labeled training data.
5. **Performance Testing:** Finally, the `test_DNN` method is used to evaluate the model. It processes the test set through the network to compute the final classification error rate by comparing the predicted labels to the ground truth.

To ensure a standard run, the following script structure is used:

```
# Example of the execution sequence
model = DNN([784, 200, 200, 200, 10]) %
train_img, train_lab = DNN.load_mnist('data/train-images-idx3-ubyte', '
    data/train-labels-idx1-ubyte') %
test_img, test_labels = DNN.load_mnist('data/t10k-images-idx3-ubyte', '
    data/t10k-labels-idx1-ubyte') %
model.pretrain_DNN(train_img, epochs=100, lr=0.01, batch_size=128) %
model.retropropagation(train_img, train_lab, epochs=200, lr=0.1,
    batch_size=128) %
error = model.test_DNN(test_img, test_labels) %
```

2 Preliminary Study: Binary AlphaDigit

Before evaluating our architecture on the MNIST benchmark, we validated the representation and generative capacities of our RBM and DBN implementations using the Binary AlphaDigit dataset.

2.1 Impact of Hidden Layer Capacity

We first investigated the relationship between the number of hidden units (q) and the RBM's generative fidelity by training the network on a subset of characters ('A' and 'B').

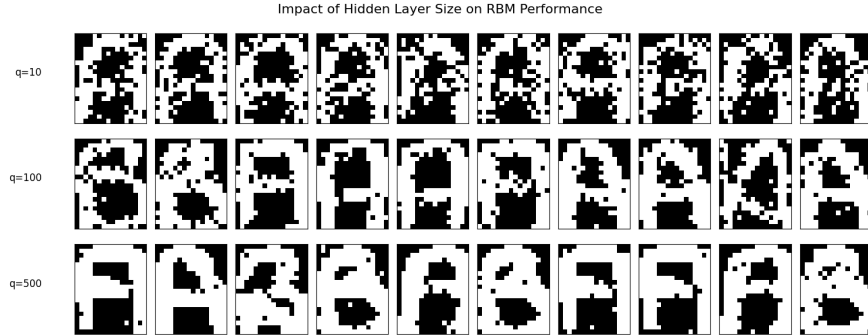


Figure 1: Generative performance of a single RBM with varying hidden layer sizes ($q = 10, 100, 500$).

The results demonstrate a direct relationship between parameter capacity and generative quality.

- **Restricted Capacity ($q = 10$):** The network acts as an informational bottleneck, lacking the parameters to fully map pixel correlations, which results in noisy and ambiguous shapes.
- **Sufficient Capacity ($q = 100$ to 500):** As the hidden layer expands, the RBM gains sufficient memory to properly separate the distinct probability distributions of the letters. This yields highly recognizable letters without high-frequency noise.

2.2 Impact of Network Depth

We then compared the generative quality of a single-layer RBM against deeper DBN architectures (2 and 3 hidden layers) trained on four characters ('A', 'B', 'C' and 'D').

Visual analysis confirms the advantage of hierarchical learning. While the 1-layer RBM produces recognizable binary images, they often contain background noise. By stacking layers, the 2-layer

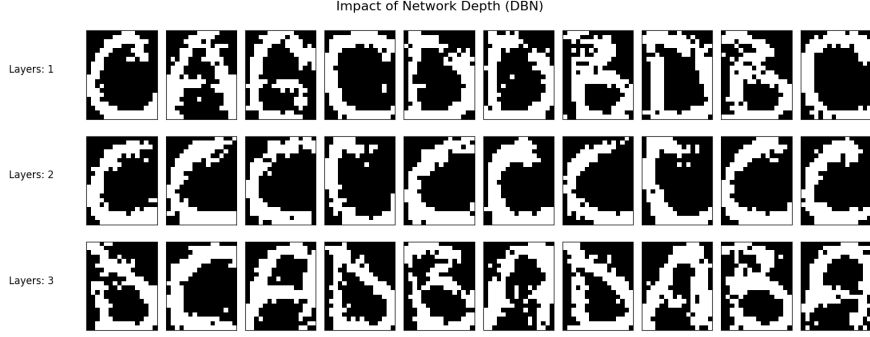


Figure 2: Visual comparison of characters generated by 1-layer, 2-layer, and 3-layer networks.

and 3-layer DBNs generate significantly smoother characters. The lower layers effectively capture local strokes, while higher layers govern the global shape. Furthermore, the 3-layer DBN shows greater generation diversity than the 2-layer model, as its larger parameter space allows it to better escape local minima.

2.3 Impact of Number of Characters

Finally, we tested the network’s generative limits by training a fixed DBN architecture ([320, 100, 100]) on sets of varying complexity: 1, 3, 10, and 36 classes.

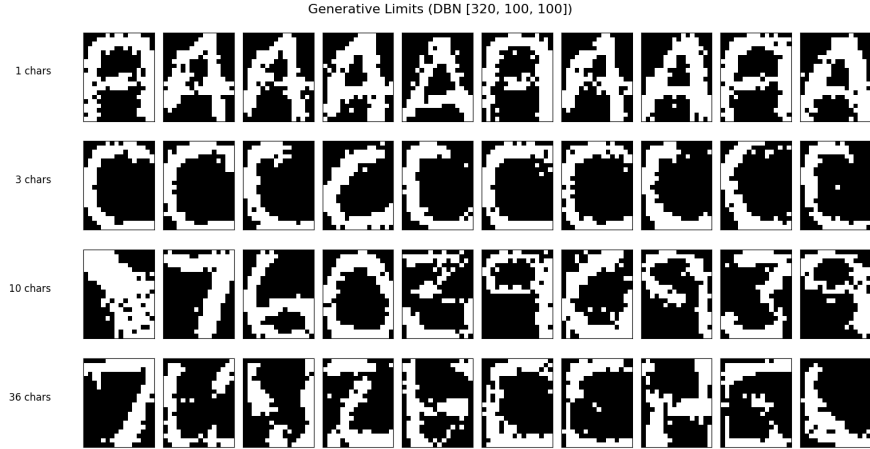


Figure 3: Generations from a fixed DBN architecture trained on 1, 3, 10, and 36 characters.

This experiment highlights capacity limitations and the phenomenon of mode blending. The network easily maps the distribution of 1 to 3 characters, generating perfect samples. However, when forced to learn 10 or 36 characters, the generated outputs degrade into unrecognizable shapes. The fixed parameter space becomes overwhelmed by the complexity of 36 distinct modes. The network begins sampling interpolations that blend multiple characters together (e.g., a mix of ‘8’, ‘B’, and ‘S’). To successfully model all 36 characters simultaneously, the network’s capacity must be increased.

3 Main Analysis: MNIST Classification

3.1 Experimental Setup

The analysis is conducted on the MNIST dataset, which is binarized at a threshold of 127. The training pipeline consists of 100 RBM epochs ($\eta = 0.01$) for unsupervised pre-training and 200 backpropagation epochs ($\eta = 0.1$) for supervised fine-tuning, using a batch size of 128.

3.2 Comparative Performance Study

3.2.1 Influence of Network Depth

We evaluated the impact of increasing the number of hidden layers from 2 to 5, maintaining a fixed width of 200 neurons per layer.

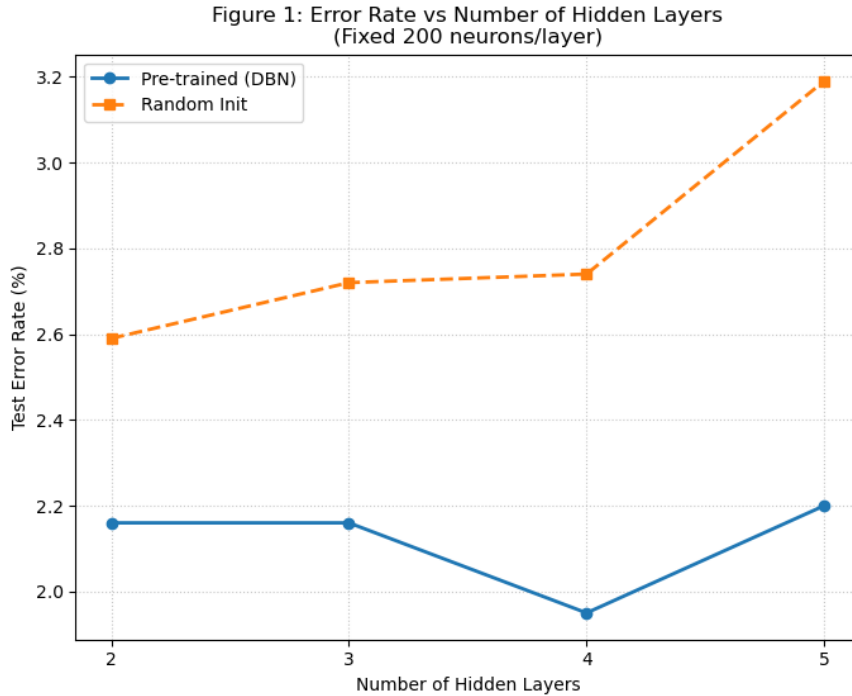


Figure 4: Test Error Rate vs. Number of Hidden Layers.

As illustrated in Figure 4, the pre-trained DBN model shows high stability, with the error rate decreasing as depth increases until dropping below 2% at 4 hidden layers, followed by a marginal increase at 5 layers. In contrast, the randomly initialized network suffers from substantial performance degradation in deeper architectures, with its error rate rising from 2.65% to 3.20%.

These empirical results confirm that unsupervised generative pre-training effectively mitigates optimization issues such as the vanishing gradient problem, providing a robust initial weight distribution that standard backpropagation alone fails to achieve in deep architectures.

Note: Across multiple training iterations, the empirical minimum error rate is consistently achieved with either 4 or 5 hidden layers, though the performance differential between these two depths remains statistically negligible.

3.2.2 Influence of Layer Capacity

Using a fixed 2-layer architecture, we varied the number of neurons per hidden layer from 100 to 700 to observe the effects of spatial representation capacity.

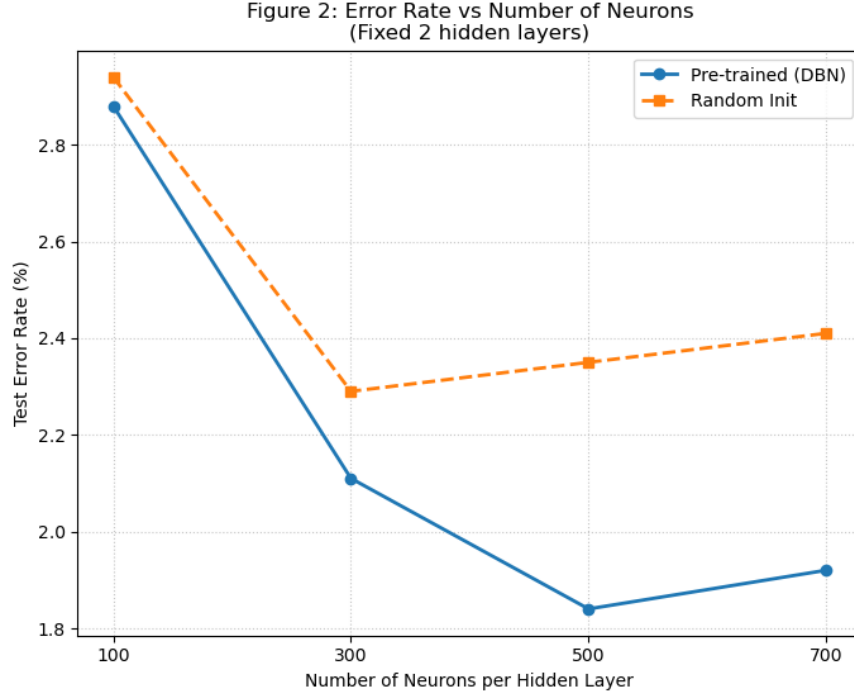


Figure 5: Test Error Rate vs. Number of Neurons per Hidden Layer.

The results in Figure 5 illustrate the performance dynamics relative to network capacity. Under a restricted capacity of 100 neurons, the DBN pre-trained model outperforms random initialization only marginally (yielding an error rate of approximately 2.70% compared to 2.75% for the random baseline). At this scale, both initialization strategies are identical, due to a severe lack of representation dimension; the restricted parameter space acts as an informational bottleneck, preventing either model from accurately capturing the complex underlying distributions of all 10 digit classes.

However, the pre-trained model demonstrates superior scalability as capacity expands. With a larger parameter space available to effectively separate the classes in the latent representation, its error rate strictly decreases, reaching its optimal performance of 1.74% at 500 neurons. Conversely, the randomly initialized model exhibits a markedly different trajectory. It reaches its best generalization early with a minimum error rate of roughly 2.45% at just 300 neurons.

Beyond their respective optima, both models experience a clear degradation in generalization when capacity is pushed to 700 neurons, with error rates rising to 1.80% for the pre-trained model and climbing back up to 2.70% for the random baseline. This trajectory confirms that while DBN pre-training is crucial for navigating the expansive parameter space of high-capacity networks, there remains a strict architectural threshold beyond which adding more neurons simply induces overfitting to the training set.

3.2.3 Influence of Training Set Size

To analyze the regularization effect of pre-training, the models were evaluated on varying subsets of the training data, ranging from 1,000 to 60,000 samples, using a fixed architecture of 2 hidden layers with 200 neurons each.

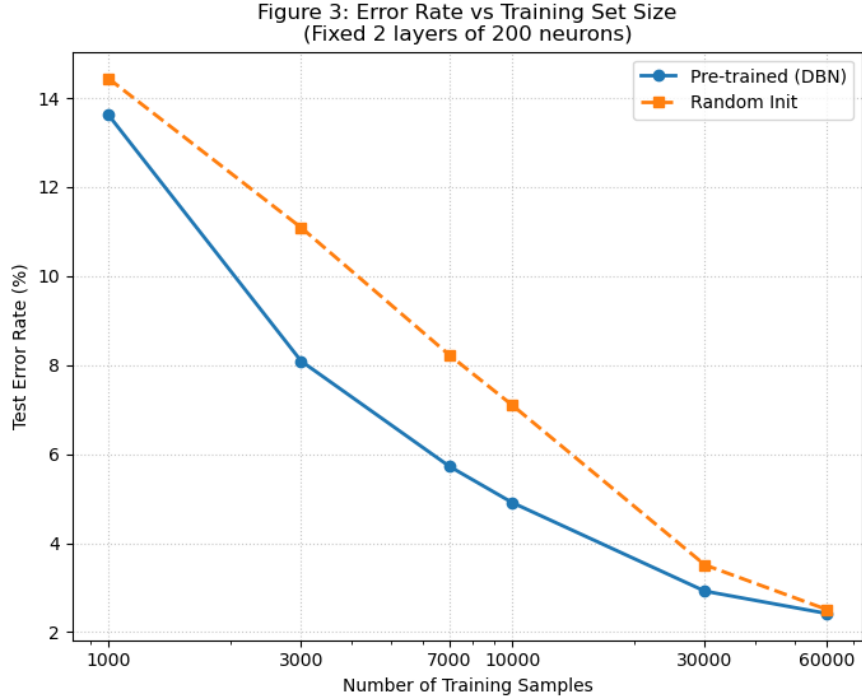


Figure 6: Test Error Rate vs. Training Set Size (Logarithmic Scale).

As depicted in Figure 6, the pre-trained architecture systematically outperforms random initialization across all data volumes. While both models have poor performance in constrained data regimes (13.62% vs. 14.44% error for 1,000 samples), the performance divergence becomes highly pronounced as the training volume increases.

For instance, when trained on 7,000 samples, the pre-trained model achieves an error rate of 5.72%, whereas the randomly initialized baseline stagnates at 8.22%. This trajectory indicates that DBN initialization functions as an effective regularizer, allowing the network to approximate the underlying data manifold more efficiently and requiring significantly fewer labeled samples to achieve strong predictive accuracy. As the training set scales to the full 60,000 samples, the performance differential attenuates, yet the pre-trained model strictly maintains its advantage with a final error rate of 2.42%, compared to 2.51% for the random baseline.

3.3 Optimal Configuration and Synthesis

Optimal Configuration

Through our empirical analysis, we identified the optimal architectural configuration to minimize the classification error rate on the MNIST benchmark.

Based on the hyperparameter space explored, the highest cross-performance was achieved using a deep architecture consisting of 4 hidden layers with 500 neurons per layer, initialized via DBN

unsupervised pre-training. This configuration successfully converged to a minimum test error rate of 1.72%.

In contrast, the optimal configuration for a network relying on random initialization achieved a best-case error rate of only **2.51%**. Notably, this baseline result was obtained using a significantly narrower architecture (2 hidden layers of 300 neurons). As demonstrated in the preceding subsections, attempts to scale the randomly initialized network to match the depth or capacity of the optimal DBN model resulted in optimization stagnation or generalization degradation.

Synthesis of Empirical Findings

The synthesis of these experiments confirms two fundamental principles regarding the training of deep neural networks:

- **Limitations of Random Initialization:** Standard random initialization, followed exclusively by supervised backpropagation, is functionally sufficient for shallow, low-capacity networks. However, it fundamentally fails to effectively optimize the expansive, non-convex parameter spaces inherent to deep and wide architectures, largely due to vanishing gradients and suboptimal local minima. To tackle this, you must reach a significant amount of data to fill the gap with pre-trained models.
- **Necessity of Generative Pre-training:** Unsupervised DBN pre-training serves as a prerequisite for leveraging the full representational power of deep neural networks. By leveraging Contrastive Divergence to capture the underlying data distribution, it positions the initial weights in a favorable region of the parameter space. This allows backpropagation to effectively fine-tune the model toward a superior local minimum.

4 Bonus: Generative Modeling Benchmark

To conclude this project, we expanded our benchmark to include modern deep generative models on the MNIST dataset. We utilized pre-implemented models and functions from PyTorch and HuggingFace for the modern architectures, comparing them against our custom Deep Belief Network (DBN).

4.1 Evaluated Architectures and Visual Comparison

We visually analyzed the generative capabilities of four distinct model families:

- **Energy-Based Models (DBN):** Our core implementation relies on Gibbs sampling. As seen in Figure 11 (top-left) the optimized DBN captures the fundamental global topologies of the ten digit classes, allowing for highly recognizable generation. However, the outputs show characteristic noise and speckling. This is a typical artifact of the independent discrete Bernoulli sampling process at the visible layer, highlighting the model’s difficulty in producing perfectly smooth, continuous strokes compared to continuous-latent architectures.
- **Variational Autoencoder (VAE):** The VAE forces a continuous, structured latent space by optimizing a lower bound on the data likelihood alongside a KL-divergence penalty. As demonstrated in Figure 11 (top-right) this results in coherent and stable digit generation. The fundamental topologies of the digits are well-captured. However, because standard VAEs optimize a pixel-wise reconstruction loss, the generated digits appear distinctly blurry with soft edges, averaging out the variance in the dataset.

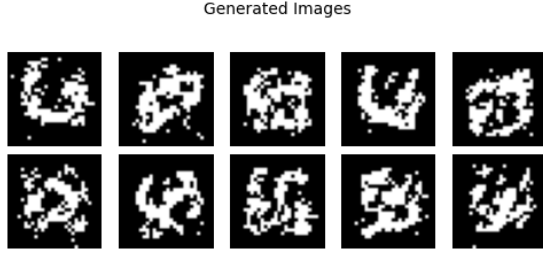


Figure 7: DBN / RBM Generations

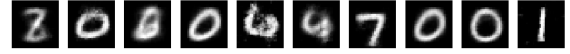


Figure 8: VAE Generations

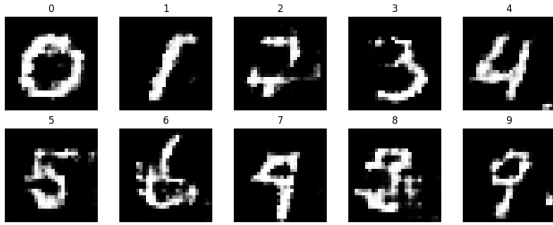


Figure 9: cGAN Generations



Figure 10: Diffusion Generations

Figure 11: Visual comparison of digits generated by models with comparable parameter counts trained on MNIST.

- **Conditional Generative Adversarial Network (cGAN):** While GANs theoretically produce sharper images by optimizing a minimax game, our implementation suffers from severe structural degradation. As shown in Figure 11 (bottom-left) the digits are heavily corrupted by high-frequency checkerboard artifacts. This is a classic architectural failure mode often caused by overlapping strides in transposed convolutions, worsened by an adversarial imbalance where the Generator produces patterned noise to exploit vulnerabilities in the Discriminator.
- **Diffusion Model:** The diffusion model, which learns to iteratively denoise a standard Gaussian distribution back into the data manifold, yields by far the highest visual fidelity. As shown in Figure 11 (bottom-right) the generated digits show both the structural coherence of the VAE and perfect sharpness. The strokes are clean, continuous, and free of the noise or artifacts that plagued the other architectures, making them practically indistinguishable from the true MNIST dataset.

4.2 Synthesis

A qualitative visual comparison of our implementations highlights the practical challenges and trade-offs inherent to each generative architecture:

1. **DBNs** are effective for unsupervised representation learning and can successfully model discrete multi-modal distributions given sufficient capacity. However, their reliance on probabilistic Gibbs sampling inherently produces noisy, speckled generations, making them less optimal for smooth, high-fidelity synthesis.
2. **cGANs** are notoriously difficult to optimize. While they bypass pixel-wise loss, they are highly

susceptible to adversarial instability and architectural artifacts (such as the checkerboard effect), requiring careful balancing of the Generator and Discriminator.

3. **VAEs** offer the most stable and reliable training process, successfully learning a continuous representation of the digits, but fundamentally trade away image sharpness to minimize their reconstruction bounds.
4. **Diffusion Models** currently represent the most robust generative framework. By systematically modeling the data distribution through a gradual Markov chain, they bypass the adversarial instability of GANs and the blurriness of VAEs, offering superior, artifact-free synthesis on the MNIST dataset.